

AWS Cloud Security Lab Report

Lab 3: VPC Security - Network Architecture, Traffic Analysis & Compliance

Prepared by: Om Fulsundar (June 2026)

Date	June 11, 2026
Platform	Amazon Web Services (AWS)
Region	eu-north-1 (Stockholm)
VPC Name	security-lab-vpc
VPC CIDR	10.0.0.0/16
Lab Type	VPC Security / Network Architecture
Severity	CRITICAL - Open Database Port (Port 3306)

1. Lab Overview

This lab is all about network-layer security in AWS - something that gets less attention than IAM and S3 misconfigurations, but is equally critical in practice. I built a two-tier VPC architecture from scratch with a public subnet hosting a web server and a private subnet simulating a database tier. I then introduced a deliberate misconfiguration - a security group with MySQL port 3306 exposed to the entire internet - and used VPC Flow Logs and AWS Config to detect and investigate it.

What makes this lab particularly practical is the combination of controls it covers: Security Groups (stateful, instance-level), Network ACLs (stateless, subnet-level), VPC Flow Logs for traffic visibility, and AWS Config for compliance detection. These four tools together form the network security monitoring stack that a real cloud security engineer would use in a production AWS environment.

Real-world relevance: Exposed database ports are one of the top causes of cloud data breaches. The MongoDB and Elasticsearch mass exposure incidents that leaked hundreds of millions of records were largely caused by databases reachable directly from the internet with no authentication - exactly the risk simulated in this lab.

2. Objectives

Here is what I set out to accomplish across the five phases of this lab:

- Design and build a two-tier VPC architecture with public and private subnets, an Internet Gateway, and correctly configured route tables.
- Create Security Groups for both the web server tier (HTTP/HTTPS/SSH) and the database tier, with the database SG intentionally misconfigured to expose port 3306 to 0.0.0.0/0.
- Configure a custom Network ACL (NACL) for the public subnet with explicit inbound and outbound rules, and understand the difference between stateful SGs and stateless NACLs.
- Enable VPC Flow Logs with CloudWatch as the destination and generate both ACCEPT and REJECT traffic to verify logging is working.
- Use AWS Config with the restricted-common-ports managed rule to detect the database SG misconfiguration as NON COMPLIANT.
- Remediate the misconfiguration by restricting the database SG source to the web-server-sg only, and validate the COMPLIANT status in AWS Config.

3. Tools and Services Used

Tool / Service	Purpose in This Lab
Amazon VPC	Core networking service used to build the isolated virtual network with subnets, route tables, and an internet gateway.
Security Groups (SGs)	Instance-level stateful firewalls controlling inbound/outbound traffic for the EC2 web server and the simulated database tier.
Network ACLs (NACLs)	Subnet-level stateless firewalls providing an additional layer of traffic control for the public subnet.
VPC Flow Logs	Captures metadata about all IP traffic flowing through the VPC network interfaces, stored in CloudWatch Logs.
Amazon CloudWatch Logs	Destination for VPC Flow Log data; used to inspect ACCEPT and REJECT entries and confirm traffic patterns.
AWS Config	Compliance monitoring service used to detect the misconfigured database security group via the restricted-common-ports rule.
EC2 (t3.micro)	Free-tier web server instance deployed in the public subnet to generate real traffic for flow log analysis.
draw.io	Used to create the VPC architecture diagram documenting the full network topology.
PowerShell / curl	Local terminal used to send HTTP requests to the EC2 instance and trigger traffic for flow log capture.

4. Lab Environment Setup

The entire lab ran on a live AWS account (Account ID: 712934828848) in the eu-north-1 (Stockholm) region. I chose Stockholm because it was a clean region with no pre-existing infrastructure, which made it easier to keep the lab resources isolated and the flow logs noise-free.

Before starting, I drew out the target architecture in draw.io to plan the component relationships: the VPC spanning both subnets, the Internet Gateway and its attachment, the two route tables (one with internet access, one without), the EC2 instance in the public subnet, and the simulated database in the private subnet. Having the architecture diagram upfront made the build phase much more systematic.

The EC2 instance I launched was a t3.micro running Amazon Linux 2 - free tier eligible. I used a simple user data script to start a basic HTTP server that returned 'Hello from Om's lab' on port 80. This gave me a real HTTP endpoint to test against when verifying the flow logs.

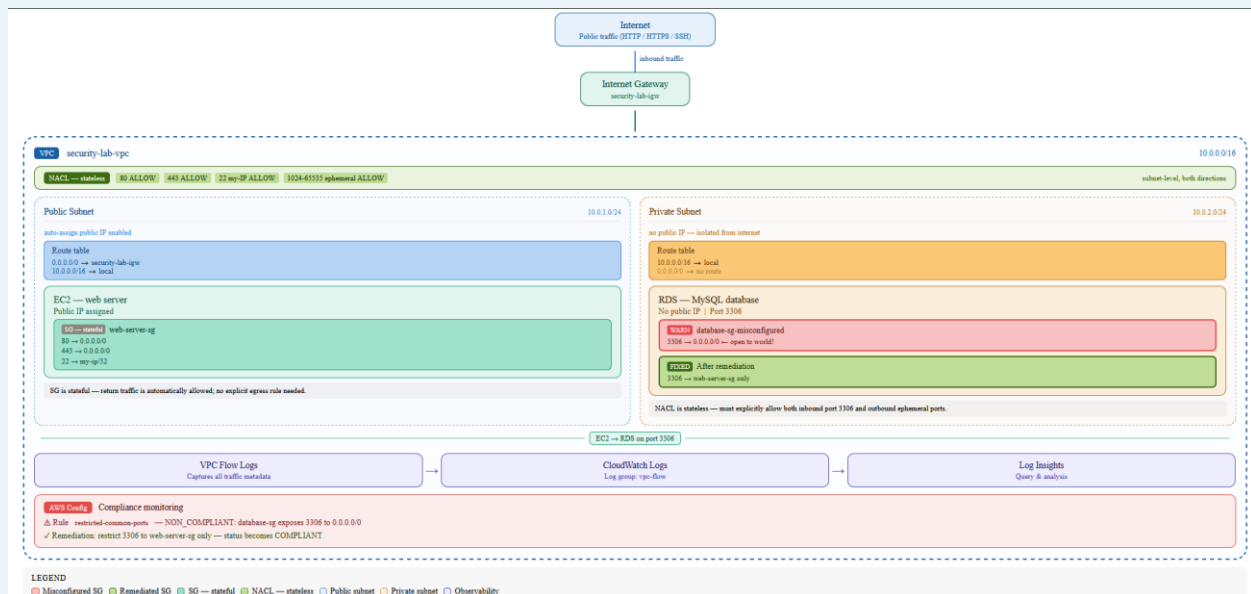
5. Step-by-Step Walkthrough

Phase 1 - VPC Architecture Design and Build

I started by creating the VPC itself: navigating to VPC > Create VPC, naming it security-lab-vpc, and assigning the CIDR block 10.0.0.0/16. This gives the VPC 65,536 IP addresses, which is far more than needed for a lab but follows the standard /16 convention for a full VPC address space.

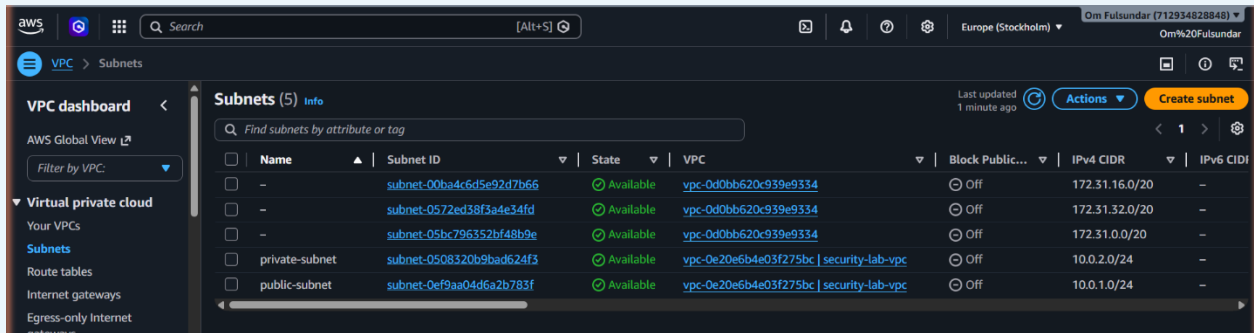
Next I created two subnets. The public subnet (10.0.1.0/24) was placed in eu-north-1a with auto-assign public IP enabled. The private subnet (10.0.2.0/24) was placed in eu-north-1b with no public IP - traffic from this subnet should never reach the internet directly. Both subnets were correctly associated with the security-lab-vpc VPC ID.

[SS1 - VPC Architecture Diagram]



draw.io architecture showing Internet -> Internet Gateway -> Public Subnet (EC2 web server, web-server-sg) -> Private Subnet (RDS MySQL, database-sg-misconfigured), VPC Flow Logs -> CloudWatch -> Log Insights, AWS Config compliance monitoring layer

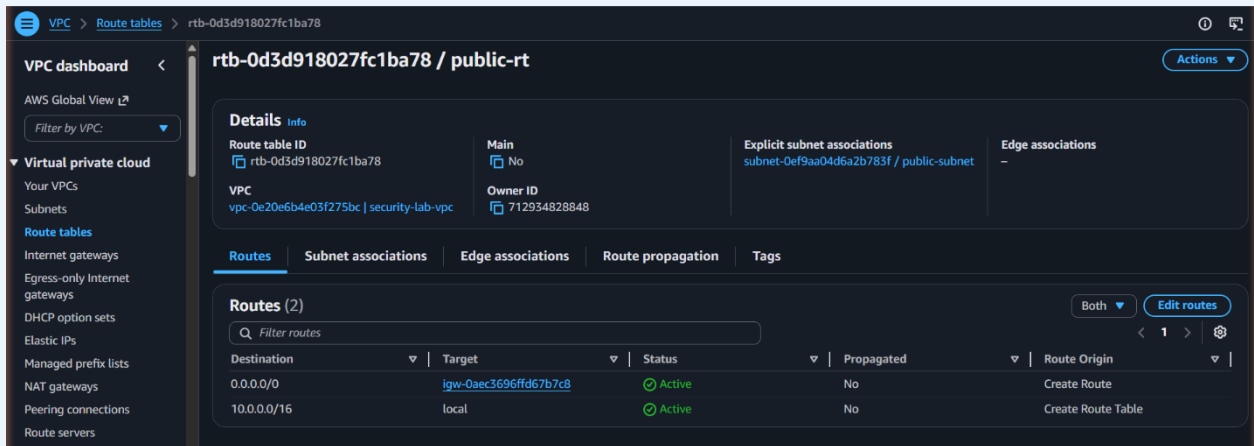
[SS2 - Subnets Console]

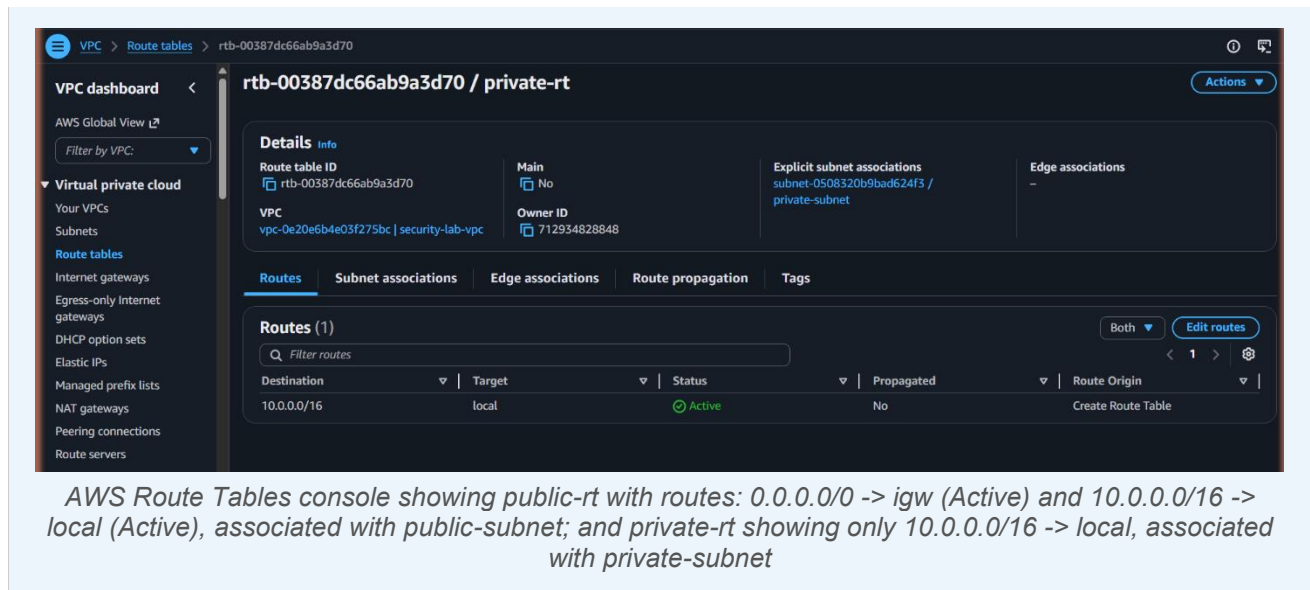


AWS VPC Subnets console showing public-subnet (10.0.1.0/24) and private-subnet (10.0.2.0/24) both available and associated with security-lab-vpc

After subnets, I created an Internet Gateway (security-lab-igw) and attached it to the VPC. Then I set up two route tables. The public route table got a route: Destination 0.0.0.0/0, Target: the Internet Gateway - this is what makes a subnet public. The private route table only has the local route (10.0.0.0/16 -> local) with no internet path.

[SS3 - Route Tables Console]





Phase 2 - Security Group Configuration (Including Misconfiguration)

I created two security groups inside security-lab-vpc. The first, web-server-sg, was correctly configured: inbound rules allowing HTTP (80) and HTTPS (443) from anywhere, and SSH (22) restricted to my IP only (117.212.252.30/32). This follows the principle of least privilege - the web server needs to be publicly reachable on web ports, but SSH should never be open to the world.

The second security group was the intentional misconfiguration. I named it database-sg-misconfigured and added a single inbound rule: MySQL/Aurora port 3306, source 0.0.0.0/0. This means any IP on the internet could attempt a connection to port 3306. In a real environment, a database should only accept connections from the application tier - never from the public internet. This is the vulnerability I then detected and remediated.

[SS4 - Security Group Misconfiguration Console]

The screenshot shows the AWS Management Console 'Create security group' page. The breadcrumb navigation is 'VPC > Security Groups > Create security group'. The page title is 'Create security group' with an 'info' link. A subtitle states: 'A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.'

Basic details

- Security group name:** database-sg-misconfigured (with an 'info' link and a note: 'Name cannot be edited after creation.')
- Description:** Database SG with open port (with an 'info' link)
- VPC:** vpc-0e20e6b4e03f275bc (security-lab-vpc) (with an 'info' link)

Inbound rules (with an 'info' link)

Type	Protocol	Port range	Source	Description - optional	
MySQL/Aurora	TCP	3306	Anywhere...		0.0.0.0/0 X

Outbound rules (with an 'info' link)

This security group has no outbound rules.

AWS Create Security Group console showing database-sg-misconfigured with inbound rule: MySQL/Aurora TCP 3306 from Anywhere (0.0.0.0/0), VPC: security-lab-vpc

An open port 3306 to the internet is a critical misconfiguration. MySQL has known authentication bypass vulnerabilities in older versions, and even patched versions can be targeted by brute-force attacks. A database port should never have 0.0.0.0/0 as its source.

Phase 3 - Network ACL Configuration

I created a custom NACL named public-subnet-nacl and associated it with the public subnet. NACLs are stateless - unlike Security Groups, they don't automatically allow return traffic. This means you have to explicitly create both inbound and outbound rules.

For inbound rules, I set: Rule 100 allowing HTTP (80) from anywhere, Rule 110 allowing HTTPS (443) from anywhere, Rule 120 allowing SSH (22) from my IP only (117.212.252.30/32), Rule 130 allowing ephemeral/return traffic on ports 1024-65535, and the default deny-all (*). For outbound, Rule 100 allows all traffic out, with a default deny.

[SS5 - NACL Rules Console]

The image displays two screenshots of the AWS Network ACLs console. The top screenshot shows the 'Inbound rules' tab for the Network ACL 'acl-0ad002590ad613147 / public-subnet-nacl'. It lists five rules: Rule 100 (HTTP 80, Allow), Rule 110 (HTTPS 443, Allow), Rule 120 (SSH 22, Allow), Rule 130 (Custom TCP 1024-65535, Allow), and a default rule (All traffic, Deny). The bottom screenshot shows the 'Outbound rules' tab for the same Network ACL, listing two rules: Rule 100 (All traffic, Allow) and a default rule (All traffic, Deny).

Top Screenshot: Inbound rules (5)

Rule number	Type	Protocol	Port range	Source	Allow/Deny
100	HTTP (80)	TCP (6)	80	0.0.0.0/0	Allow
110	HTTPS (443)	TCP (6)	443	0.0.0.0/0	Allow
120	SSH (22)	TCP (6)	22	117.212.252.30/32	Allow
130	Custom TCP	TCP (6)	1024 - 65535	0.0.0.0/0	Allow
*	All traffic	All	All	0.0.0.0/0	Deny

Bottom Screenshot: Outbound rules (2)

Rule number	Type	Protocol	Port range	Destination	Allow/Deny
100	All traffic	All	All	0.0.0.0/0	Allow
*	All traffic	All	All	0.0.0.0/0	Deny

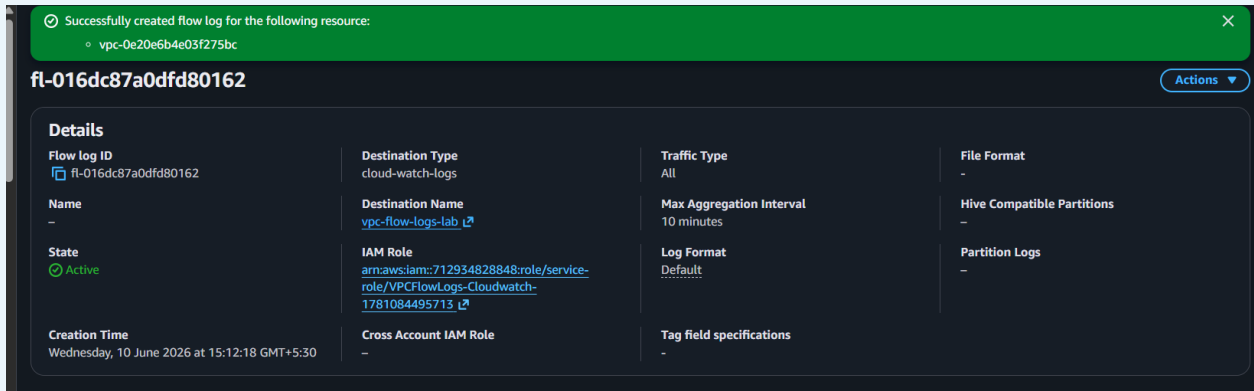
*AWS Network ACLs console showing public-subnet-nacl inbound rules: 100 HTTP Allow, 110 HTTPS Allow, 120 SSH Allow (my IP), 130 Custom TCP 1024-65535 Allow, * Deny all; and outbound rules: 100 All traffic Allow, * Deny all*

The most important thing I took from this phase: forgetting Rule 130 (the ephemeral port range) in the NACL inbound rules would have broken all outbound-initiated traffic. HTTP responses flow back on high ports - if the NACL doesn't allow those return packets in, sessions silently fail. This is the most common NACL configuration mistake and it's easy to make because Security Groups handle it automatically.

Phase 4 - VPC Flow Logs Setup

I navigated to VPC > security-lab-vpc > Flow Logs and created a new flow log. The configuration: Filter set to All (capturing both ACCEPT and REJECT), destination set to CloudWatch Logs, log group name vpc-flow-logs-lab, and a new IAM role (VPCFlowLogs-Cloudwatch-1781084495713) created automatically to grant the VPC permission to write to CloudWatch. The flow log was created successfully with ID fl-016dc87a0dfd80162 and immediately showed as Active.

[SS6 - VPC Flow Logs Setup]



AWS VPC Flow Logs console showing fl-016dc87a0dfd80162 Active, Destination Type: cloud-watch-logs, Destination Name: vpc-flow-logs-lab, Traffic Type: All, IAM Role: VPCFlowLogs-Cloudwatch-1781084495713, Creation Time: June 10, 2026

Phase 5 - Traffic Generation and Flow Log Analysis

With the EC2 instance running on its public IP (16.16.144.226), I sent an HTTP request from my local PowerShell terminal using curl. The response came back with status 200 OK and content 'Hello from Om's lab' - confirming the web server was running and the Security Group and NACL rules were allowing the traffic through correctly.

```
PS C:\Users\Lenovo> curl http://16.16.144.226

StatusCode      : 200
StatusDescription : OK
Content         : Hello from Om's lab
RawContent      : HTTP/1.1 200 OK
                  Keep-Alive: timeout=5, max=100
                  Connection: Keep-Alive
                  Content-Type: text/html; charset=UTF-8
```

After waiting about 10 minutes for the logs to propagate, I opened CloudWatch > Log Groups > vpc-flow-logs-lab and found the log stream for my EC2 network interface. The ACCEPT entries were visible - each line showing the source IP, destination IP (10.0.1.53), port 80, protocol 6 (TCP), and the action ACCEPT OK. This confirmed the flow logging was capturing legitimate allowed traffic correctly.

[SS7 - CloudWatch ACCEPT Entries + Terminal Output]

Log events

Actions

Start tailing

Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Q

Filter events - press enter to search

Clear

1m

30m

1h

12h

Custom

UTC timezone

Display

Timestamp	Message
2026-06-10T11:48:44.000Z	2 712934828848 eni-0573c657a95fe220b 66.132.172.119 10.0.1.53 38783 25583 6 1 60 1781092124 1781092143 REJECT OK
2026-06-10T11:48:44.000Z	2 712934828848 eni-0573c657a95fe220b 85.217.140.25 10.0.1.53 35174 12531 6 1 52 1781092124 1781092143 REJECT OK
2026-06-10T11:49:11.000Z	2 712934828848 eni-0573c657a95fe220b 59.97.218.174 10.0.1.53 56759 80 6 5 370 1781092151 1781092181 ACCEPT OK
2 712934828848 eni-0573c657a95fe220b 59.97.218.174 10.0.1.53 56759 80 6 5 370 1781092151 1781092181 ACCEPT OK	
2026-06-10T11:49:11.000Z	2 712934828848 eni-0573c657a95fe220b 152.32.156.138 10.0.1.53 36013 11389 6 1 44 1781092151 1781092181 REJECT OK
2026-06-10T11:49:11.000Z	2 712934828848 eni-0573c657a95fe220b 179.82.48.44 10.0.1.53 33275 48190 17 1 61 1781092151 1781092181 REJECT OK
2026-06-10T11:49:11.000Z	2 712934828848 eni-0573c657a95fe220b 10.0.1.53 59.97.218.174 80 56759 6 4 496 1781092151 1781092181 ACCEPT OK
2 712934828848 eni-0573c657a95fe220b 10.0.1.53 59.97.218.174 80 56759 6 4 496 1781092151 1781092181 ACCEPT OK	
2026-06-10T11:49:11.000Z	2 712934828848 eni-0573c657a95fe220b 66.132.186.41 10.0.1.53 23682 49152 6 1 60 1781092151 1781092181 REJECT OK
2026-06-10T11:49:11.000Z	2 712934828848 eni-0573c657a95fe220b 219.151.190.165 10.0.1.53 52154 8880 6 1 40 1781092151 1781092181 REJECT OK
2026-06-10T11:49:11.000Z	2 712934828848 eni-0573c657a95fe220b 78.128.112.30 10.0.1.53 52402 42053 6 1 40 1781092151 1781092181 REJECT OK

```
PS C:\Users\Lenovo> curl http://16.16.144.226

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription : OK
Content          : Hello from Om's lab

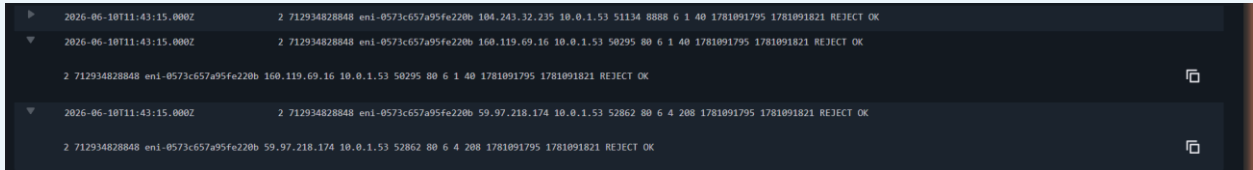
RawContent       : HTTP/1.1 200 OK
                  Keep-Alive: timeout=5, max=100
                  Connection: Keep-Alive
                  Accept-Ranges: bytes
                  Content-Length: 20
                  Content-Type: text/html; charset=UTF-8
                  Date: Wed, 10 Jun 2026 11:32:16 GMT
                  ETag: "1..."
Forms            : {}
Headers          : {[Keep-Alive, timeout=5, max=100], [Connection, Keep-Alive], [Accept-Ranges, bytes], [Content-Length, 20]...}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 20

PS C:\Users\Lenovo>
```

CloudWatch log stream showing ACCEPT OK entries for port 80 traffic to 10.0.1.53, alongside PowerShell terminal showing curl HTTP 200 response with content 'Hello from Om's lab'

To generate REJECT entries, I ran connection tests against ports that are not open on the EC2 instance - ports 21 (FTP), 25 (SMTP), and 3389 (RDP). The Security Group and NACL both block these ports, so the flow logs recorded REJECT entries for each attempt.

[SS8 - CloudWatch REJECT Entries]



▶	2026-06-10T11:43:15.000Z	2 712934828848 eni-0573c657a95fe220b 104.243.32.235 10.0.1.53 51134 8888 6 1 40 1781091795 1781091821 REJECT OK	
▼	2026-06-10T11:43:15.000Z	2 712934828848 eni-0573c657a95fe220b 160.119.69.16 10.0.1.53 50295 80 6 1 40 1781091795 1781091821 REJECT OK	
		2 712934828848 eni-0573c657a95fe220b 160.119.69.16 10.0.1.53 50295 80 6 1 40 1781091795 1781091821 REJECT OK	📄
▼	2026-06-10T11:43:15.000Z	2 712934828848 eni-0573c657a95fe220b 59.97.218.174 10.0.1.53 52862 80 6 4 208 1781091795 1781091821 REJECT OK	
		2 712934828848 eni-0573c657a95fe220b 59.97.218.174 10.0.1.53 52862 80 6 4 208 1781091795 1781091821 REJECT OK	📄

CloudWatch log stream showing REJECT OK entries for traffic from external IPs to 10.0.1.53 on various blocked ports, confirming Security Group and NACL deny rules are working and being logged

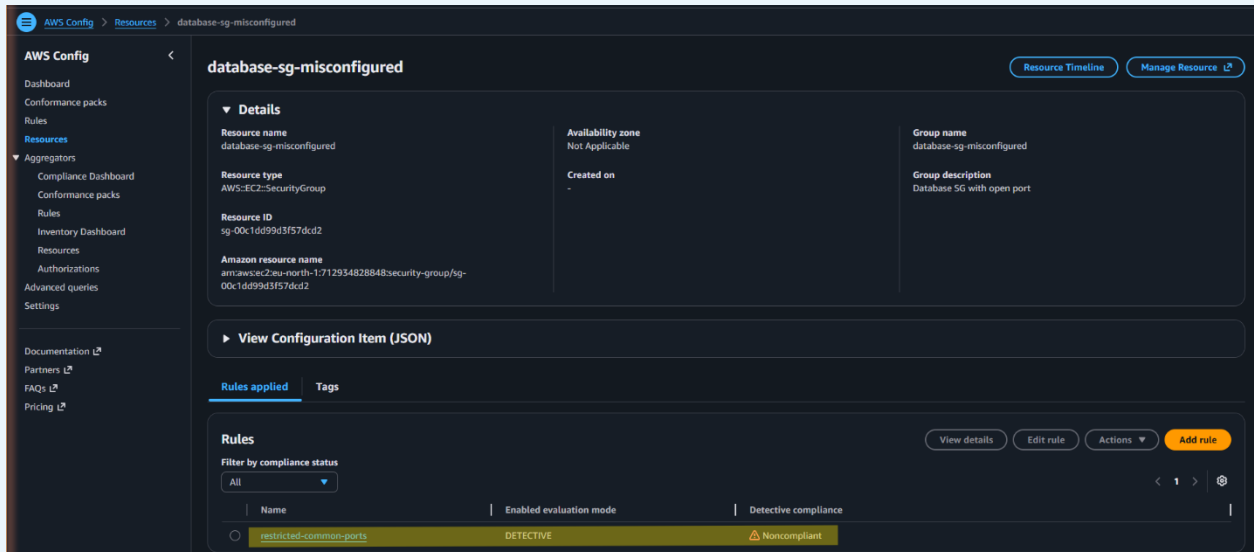
REJECT entries in flow logs are a powerful detection signal. A burst of REJECTs across many ports from a single source IP is a strong indicator of a port scan - exactly the kind of reconnaissance an attacker runs before attempting an exploit. In a production SOC, this pattern would trigger an alert.

Phase 6 - AWS Config Detection

I opened AWS Config > Rules and added the AWS-managed rule restricted-common-ports. This rule flags security groups that allow unrestricted inbound access on commonly abused ports including 3306 (MySQL), 1433 (MSSQL), 5432 (PostgreSQL), 3389 (RDP), and 22 (SSH). It's mapped to security frameworks like NIST-SP-800-53-r5 and CIS-AWS benchmarks.

On running the evaluation, database-sg-misconfigured was immediately flagged as NON COMPLIANT. The resource detail view in Config showed the security group resource name, its ARN, and the rules-applied panel with restricted-common-ports in orange NON COMPLIANT status. The finding was exactly what we expected: port 3306 open to the world is a textbook compliance violation.

[SS9 - AWS Config NON-COMPLIANT Finding]



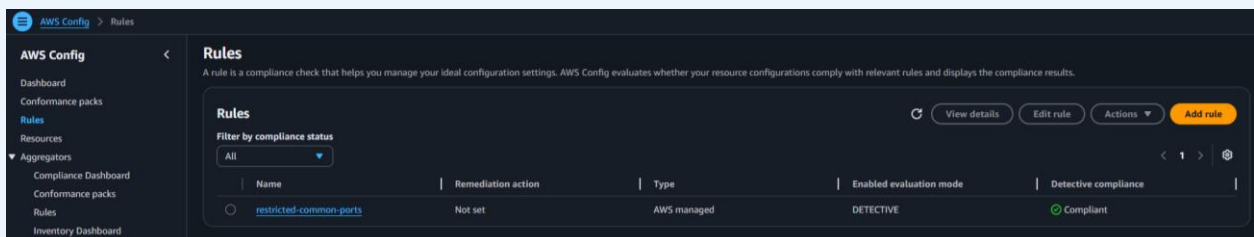
AWS Config Resources console showing database-sg-misconfigured (AWS::EC2::SecurityGroup) with restricted-common-ports rule in Noncompliant status under Rules Applied tab

Phase 7 - Remediation and Validation

Remediation involved editing the inbound rule of database-sg-misconfigured. I changed the source from 0.0.0.0/0 to the security group ID of web-server-sg. This is the correct pattern for a tiered architecture: the database only accepts connections from the application layer, not from the internet. No IP addresses, no CIDR blocks - just a security group reference, which means only instances running with that SG can connect.

After saving the rule change, I went back to AWS Config and triggered a re-evaluation of the restricted-common-ports rule. The result changed from Noncompliant to Compliant - the green status confirming the database SG was no longer exposing port 3306 to the world.

[SS10 - AWS Config COMPLIANT After Remediation]



AWS Config Rules list showing restricted-common-ports with green Compliant status after database-sg-misconfigured inbound rule was changed from 0.0.0.0/0 to web-server-sg source only

6. Findings

Finding	Security Group database-sg-misconfigured allowed inbound MySQL traffic (port 3306) from any IP on the internet (0.0.0.0/0).
Vulnerability Type	Overly Permissive Security Group / Exposed Database Port
Severity	CRITICAL
CVSS-equivalent	High - Network accessible, no authentication bypass required to attempt connection
Affected Resource	arn:aws:ec2:eu-north-1:712934828848:security-group/sg-00c1dd99d3f57dcd2
SG Name	database-sg-misconfigured
Exposed Port	3306 (MySQL/Aurora) - Source: 0.0.0.0/0
Region	eu-north-1 (Stockholm)
Config Rule Triggered	restricted-common-ports (NON COMPLIANT)
Flow Log Evidence	REJECT entries captured in vpc-flow-logs-lab for blocked port scan traffic

Real-World Impact

If a real RDS or self-managed MySQL instance had been attached to database-sg-misconfigured, the consequences could include:

- Direct brute-force attacks against MySQL authentication from anywhere on the internet - no VPN, no authentication proxy, nothing between the attacker and the database login prompt.
- Exploitation of any unpatched MySQL CVEs reachable over port 3306, potentially enabling unauthenticated remote code execution.
- Exfiltration of all database contents if valid credentials were obtained - via password reuse from another breach, default credentials, or weak passwords.
- Compliance violations under PCI-DSS, HIPAA, and GDPR, all of which mandate that database systems not be directly internet-accessible.
- Lateral movement opportunities: once inside the database server, an attacker can enumerate the private subnet and attempt to reach other internal resources.

7. Detection Method

Detection in this lab used two complementary mechanisms:

VPC Flow Logs - Traffic Visibility

1. VPC Flow Logs captured all IP traffic at the network interface level, recording source IP, destination IP, port, protocol, bytes, packets, and the action (ACCEPT or REJECT).

2. ACCEPT entries confirmed legitimate HTTP traffic was flowing correctly to port 80 on the web server EC2 instance.
3. REJECT entries confirmed the Security Group and NACL were blocking traffic on non-permitted ports. In a real environment, a burst of REJECT entries from a single source IP across many ports would indicate a port scan and trigger a SOC alert.
4. Flow logs also captured inter-subnet traffic, making it possible to audit whether any unexpected communication paths exist between the public and private subnets.

AWS Config - Compliance Detection

5. The restricted-common-ports AWS Config managed rule evaluated all security groups in the account against a list of high-risk ports.
6. The evaluation flagged database-sg-misconfigured as NON COMPLIANT because port 3306 was open to 0.0.0.0/0, which matches the rule's detection criteria.
7. The Config finding provides a clear, auditable record of the misconfiguration - including the resource ARN, creation timestamp, and compliance status history.

In a production environment, these two detection signals should feed into automated workflows. A Lambda function could auto-remediate Config findings by revoking overly permissive SG rules. A CloudWatch metric filter on REJECT entries could trigger an SNS alert to the security team when port scan patterns are detected.

8. Remediation

Step 1 - Edit the Database Security Group Inbound Rule

In EC2 > Security Groups > database-sg-misconfigured > Inbound rules, I edited the MySQL/Aurora rule. Changed the Source from 0.0.0.0/0 (Anywhere) to the Security Group ID of web-server-sg. This is the security-group-as-source pattern - only EC2 instances that are members of web-server-sg can initiate a connection to port 3306. The internet cannot.

Step 2 - Re-Evaluate in AWS Config

Triggered a manual re-evaluation of the restricted-common-ports rule in AWS Config. The database-sg-misconfigured resource changed status from Noncompliant to Compliant, confirming the fix was detected correctly.

Before vs. After Comparison

Before Remediation	After Remediation
Port 3306 open to 0.0.0.0/0 (entire internet)	Port 3306 restricted to web-server-sg source only
Any IP can attempt MySQL authentication	Only web server EC2 instances can reach the database port
AWS Config: NON COMPLIANT	AWS Config: COMPLIANT
Internet → database: possible connection	Internet → database: blocked at SG level
No network-layer isolation between tiers	Proper tiered isolation: internet can only reach web tier

9. Key Learnings

- Security Groups and NACLs serve different purposes and both matter. SGs are stateful and instance-level - ideal for application-aware rules. NACLs are stateless and subnet-level - useful for blocking broad traffic categories before it even reaches instances. Forgetting to allow ephemeral return traffic (1024-65535) in the NACL was a real gotcha that would have silently broken HTTP responses.
- The security-group-as-source pattern is the correct way to build tiered architectures. Referencing web-server-sg as the source for port 3306 means the rule automatically tracks which instances are in that SG. No IP address management, no CIDR maintenance - just a logical reference that scales correctly as instances come and go.
- VPC Flow Logs are invaluable for forensics but need to be enabled before you need them. The REJECT entries I saw during this lab would, in a real environment, be the first indicator of a port scan or reconnaissance activity. Without flow logs, that signal is completely invisible. Like CloudTrail in Lab 1, this is a control that only works if it was already running when the incident happened.
- What surprised me most: the volume of unsolicited traffic hitting a freshly launched EC2 instance. Within minutes of the instance getting a public IP, REJECT entries started appearing in the flow logs from IPs I had never contacted. Automated scanners and bots constantly probe the internet for open ports - this is the reality that every public-facing AWS resource faces the moment it gets a public IP address.
- AWS Config gives you compliance evidence, not just a finding. The resource timeline and compliance history in Config create an audit trail showing exactly when a resource was compliant or non-compliant. This is what you'd use to demonstrate to an auditor that a finding was identified and remediated - not just a screenshot, but a timestamped record in the AWS console.

10. Conclusion

This lab brought together the largest set of AWS security controls covered in the series so far. Building the VPC architecture from scratch - subnets, route tables, Internet Gateway, Security Groups, NACLs - gave me a much clearer picture of how network traffic actually flows in AWS and where each control layer sits. The misconfiguration (port 3306 open to the world) was simple to create, but the detection and remediation cycle reinforced exactly why defense in depth matters.

VPC Flow Logs ended up being the most genuinely interesting part of this lab. Watching ACCEPT entries appear after my curl request and REJECT entries pile up from unsolicited scanning bots made the network security concepts concrete in a way that reading about them doesn't. It's one thing to know that Security Groups block traffic. It's another to see the REJECT entries timestamped in CloudWatch and know that those are real connection attempts being stopped.

For cloud security and SOC roles, the skills from this lab - reading flow log entries, correlating SG and NACL rules, using Config to detect misconfigured network controls, and knowing how to correctly implement security-group-as-source tiering - are directly applicable to the kinds of investigations and remediations that come up in real cloud environments every week.

Key takeaway: In AWS, network security is not a single control - it's a stack of layers. Security Groups handle instance-level traffic. NACLs handle subnet-level traffic. Flow Logs make all of it visible. Config makes it auditable. Any one layer alone is insufficient; the combination is what creates a defensible architecture.